5.3 Attention scoring functions

- in the previous section, we used a Gaussian kernel to model interactions between queries and keys
- treating the exponent of the Gaussian kernel in (4) as an *attention scoring function* (or *scoring function*, for short), the results of this function were essentially fed into a softmax operation
- as a result, we obtained a probability distribution (attention weights) over values that are paired with keys
- in the end, the output of the attention pooling is simply a weighted sum of the values based on these attention weights

5.3 Attention scoring functions

- at a high level, we can use the above algorithm to instantiate the framework of attention mechanisms in Figure 3
- denoting an attention scoring function by a, Figure 4 illustrates how the output of attention pooling can be computed as a weighted sum of values
- since attention weights are a probability distribution, the weighted sum is essentially a weighted average

5.3 Attention scoring functions

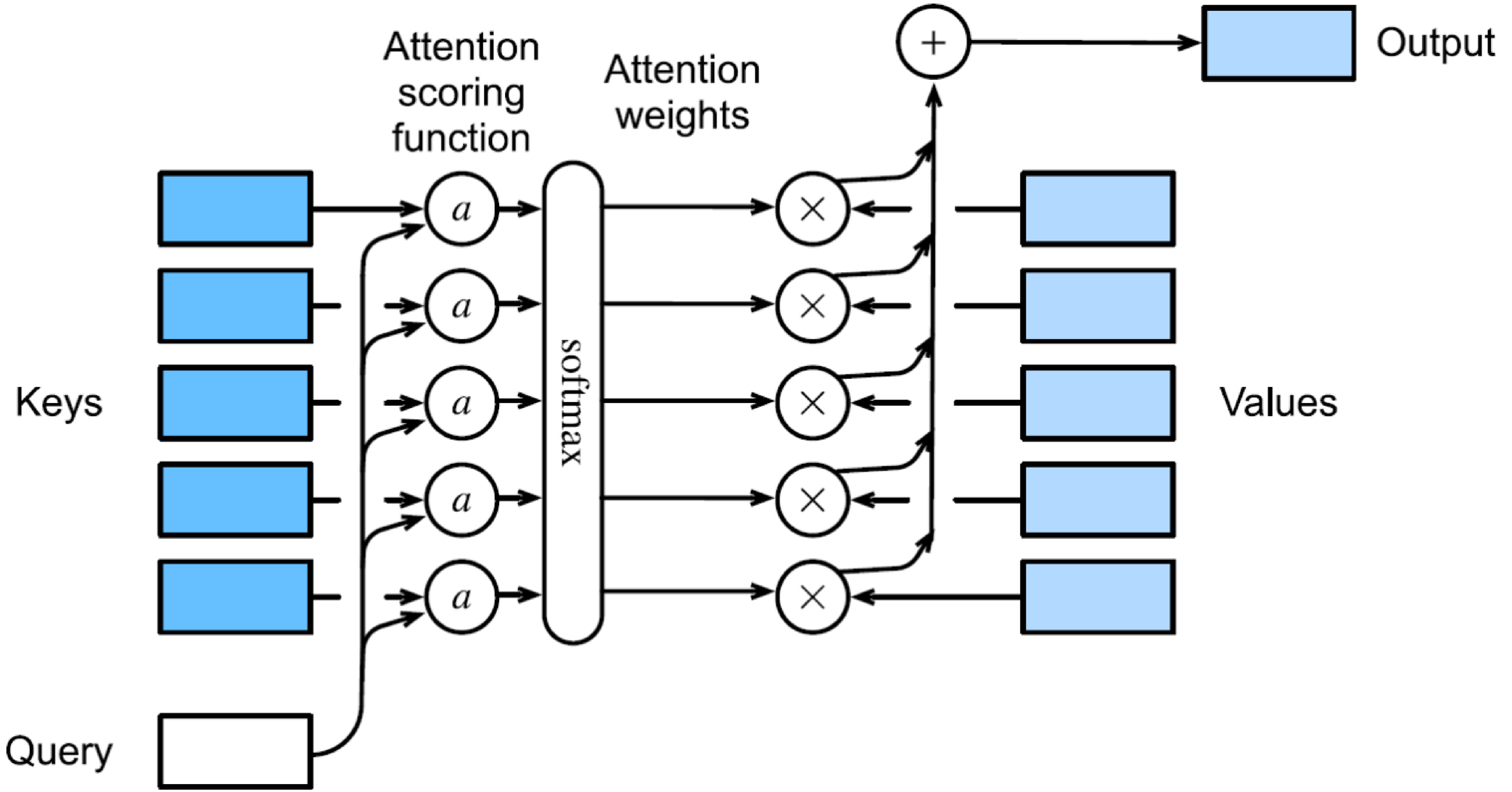


Figure 4: Computing the output of attention pooling as a weighted average of values.

5.3 Attention scoring functions

- mathematically, suppose that we have a query $\boldsymbol{q} \in \mathbb{R}^q$ and m key-value pairs $(\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)$, where any $\boldsymbol{k}_i \in \mathbb{R}^k$ and any $\boldsymbol{v}_i \in \mathbb{R}^v$
- the attention pooling f is instantiated as a weighted sum of the values:

$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i) \boldsymbol{v}_i \in \mathbb{R}^v, \tag{5}$$

where the attention weight (scalar) for the query $\boldsymbol{q}$ and key $\boldsymbol{k}_i$ is computed by the softmax operation of an attention scoring function a that maps two vectors to a scalar:

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \text{softmax}(a(\boldsymbol{q}, \boldsymbol{k}_i)) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))} \in \mathbb{R}. \tag{6}$$

- as we can see, different choices of the attention scoring function a lead to different behaviors of attention pooling
- in this section, we introduce two popular scoring functions that we will use to develop more sophisticated attention mechanisms later

5.3 Attention scoring functions

- in general, when queries and keys are vectors of different lengths, we can use additive attention as the scoring function
- given a query $\boldsymbol{q} \in \mathbb{R}^q$ and a key $\boldsymbol{k} \in \mathbb{R}^k$, the *additive attention* scoring function is defined as:

$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{w}_v^\top \tanh(\boldsymbol{W}_q \boldsymbol{q} + \boldsymbol{W}_k \boldsymbol{k}) \in \mathbb{R}, \tag{7}$$

  where the parameters $\boldsymbol{W}_q \in \mathbb{R}^{h \times q}$, $\boldsymbol{W}_k \in \mathbb{R}^{h \times k}$, and $\boldsymbol{w}_v \in \mathbb{R}^h$ are learnable
- equivalent to (7), the query and the key are concatenated and fed into an MLP with a single hidden layer whose number of hidden units is h, a hyperparameter, and is implemented by using tanh as the activation function and disabling bias terms

5.3 Attention scoring functions

- a more computationally efficient design for the scoring function can be simply the *dot product*
- however, the dot product operation requires that both the query and the key have the same vector length, say d
- assume that all the elements of the query and the key are independent random variables with zero mean and unit variance
- the dot product of both vectors has zero mean and a variance of d

5.3 Attention scoring functions

- to ensure that the variance of the dot product still remains one, regardless of vector length, the *scaled dot-product attention* scoring function:

$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{q}^\top \boldsymbol{k} / \sqrt{d}$$

  divides the dot product by $\sqrt{d}$
- in practice, we often think in mini-batches for efficiency, such as computing attention for n queries and m key-value pairs, where queries and keys are of length d, and values are of length v
- the scaled dot-product attention of queries $\boldsymbol{Q} \in \mathbb{R}^{n \times d}$, keys $\boldsymbol{K} \in \mathbb{R}^{m \times d}$, and values $\boldsymbol{V} \in \mathbb{R}^{m \times v}$ is:

$$\text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}\right)\boldsymbol{V} \in \mathbb{R}^{n \times v}. \tag{8}$$